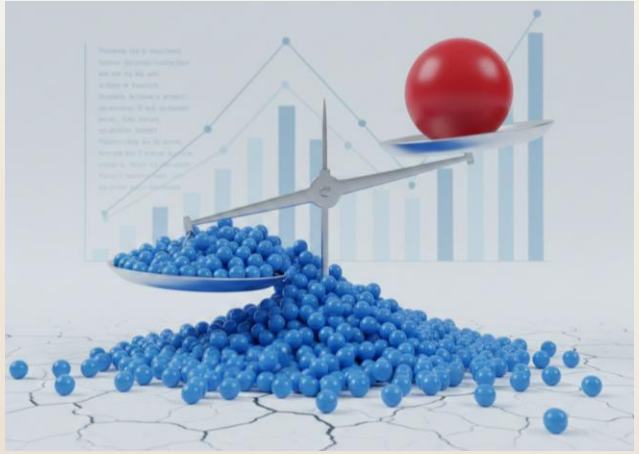
Handling Class Imbalance with SMOTE



What is Class Imbalance?

A dataset where one class (the majority) vastly outnumbers another (the minority) is said to be ***class-imbalanced***. In many real-world classification problems, one class is far more common than another.

- **Fraud detection:** 0.1% of transactions are fraudulent
- **Medical diagnosis:** 1–5% of patients have a rare disease
- **Spam filtering:** Spam may be a small fraction of all email
- **Network intrusion detection**: Malicious traffic is a tiny fraction of all network packets.
- **Defect detection in manufacturing**: Most manufactured items are defect-free; defective cases may be <0.5%.
- **Wildlife conservation / species detection**: Camera traps show animals in only a small fraction of images; rare species appear in <1%.
- **Earthquake early warning**: True seismic events are extremely rare compared to non-events in continuous sensor data.
- **Credit default prediction**: Defaults occur in only a small % of loans.
- **Coastal water quality monitoring:** Hypoxic events (dissolved oxygen ≤2.0 mg/L) are rare relative to normal (normoxic) conditions, creating a highly imbalanced dataset for binary classification.

What does Class Imbalance Matter?

A classifier trained on imbalanced data is biased toward the majority class. Consider a dataset with 95% positive and 5% negative examples:

Strategy	Accuracy
Predict everything as positve	95%
Actually learn the minority class	Harder, but more useful

High accuracy can be misleading.

We saw this previously in *Evaluation Metrics*: A classifier that predicts only the majority class gets:

- High accuracy,
- Recall of the minority class is 0,
- Near-0 MCC.

The minority class is often the most important one to get right.

That's exactly why Recall and F1 exist missing a rare but critical case can be far more costly than a false alarm.

Standard learning algorithms optimize *overall* error.

That is equivalent to maximizing accuracy!

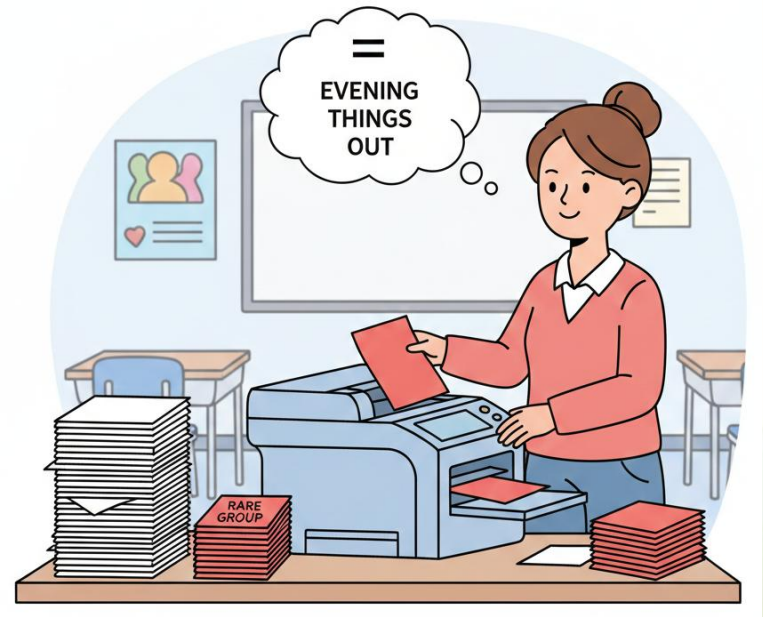
the metric we just agreed is broken under imbalance.

Possible Solutions

Two straightforward strategies address imbalance by changing the training set:

Oversample the Minority Class: Duplicate minority class examples until classes are balanced

- Creates a balanced training set
- The risk is Overfitting: the model memorizes the same minority examples.



Undersample the Majority Class: Remove majority class examples until classes are balanced

- Creates a balanced training set
- We risk losing potentially useful information from the majority class

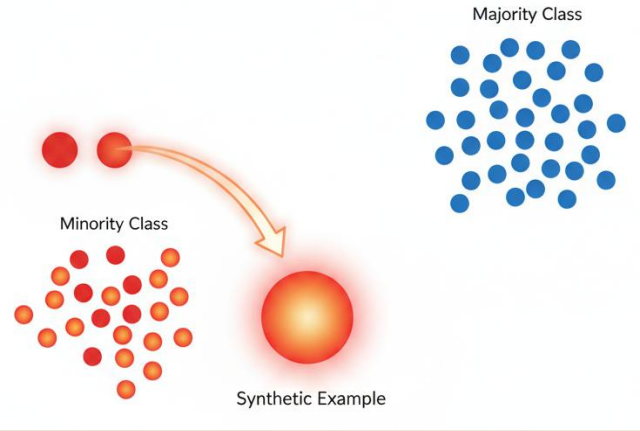
Both approaches are simple but come with real trade-offs. Can we do better?

Synthetic Oversampling with SMOTE

Instead of duplicating minority examples, what if we generated new, synthetic ones?

- New examples should be similar to real minority instances
- They should diversify the minority class, not just repeat it

This addresses the overfitting risk of naive oversampling.



Synthetic Minority Over-sampling TEchnique (SMOTE)

Chawla et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research.

Interpolation: Motivation

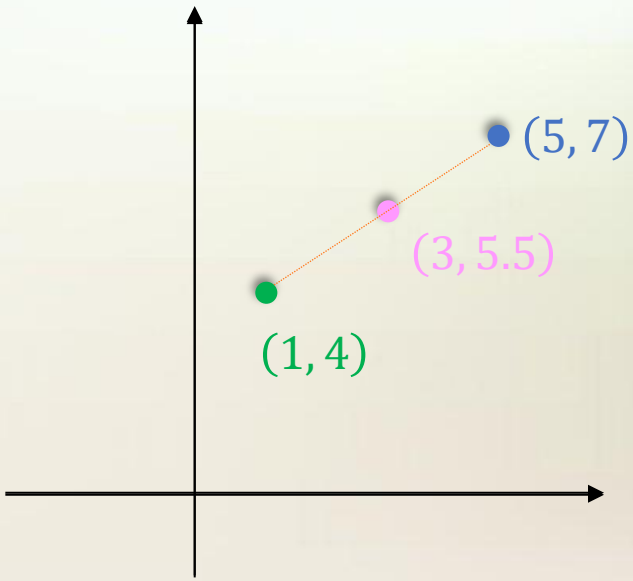
Interpolation is a ***blending*** of two values.

Consider two points in the Euclidean Plane.

If we move from $(1, 4)$ to $(5, 7)$, using a linear function we might have to compute the equation of the line using point-slope form:

$$\mathrm{y} - y_0 = \frac{y_1 - y_0}{x_1 - x_0}(\mathrm{x} - x_0).$$

For the two points, $\mathrm{y} - 7 = \frac{4-7}{1-5}(\mathrm{x} - 5) \leftrightarrow \mathrm{y} - 7 = \frac{3}{4}(\mathrm{x} - 5)$.



We may then consider any x-value in the interval $[1, 5]$.

For example, we might compute the midpoint as:

$$(x_M, y_M) = \left(\frac{1+5}{2}, f\left(\frac{1+5}{2}\right)\right) = (3, 5.5)$$

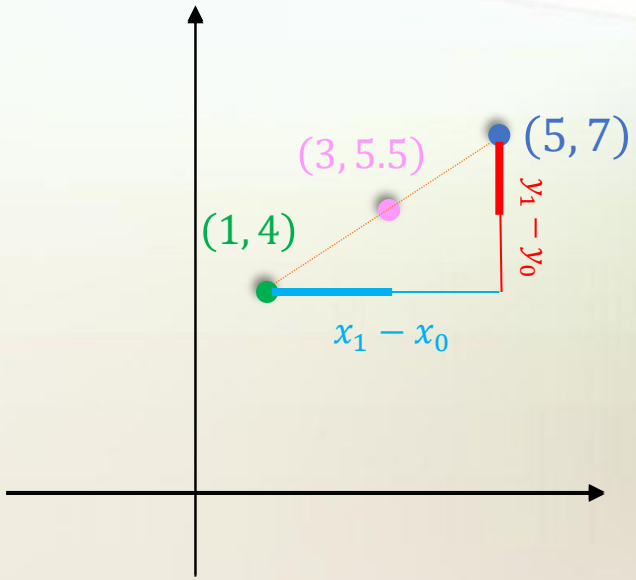
where f is the equation of the line.

Interpolation: Motivation II

To acquire a blending of points, we do not want to compute the function f that passes through the two points, we want to compute individual points along that line.

Consider again computing the midpoint of the two points.
We are well-aware of a formula for doing so:

$$(x_M, y_M) = \left(\frac{x_0 + x_1}{2}, \frac{y_0 + y_1}{2}\right) = \left(\frac{1+5}{2}, \frac{4+7}{2}\right) = (3, 5.5)$$



This technique is very specific to a single point. We should work to generalize this notion to allow us to compute any point on the line segment between our given points.

$$(x_M, y_M) = \left(x_0 + 0.5 * (x_1 - x_0), y_0 + 0.5 * (y_1 - y_0)\right)$$

For this example,

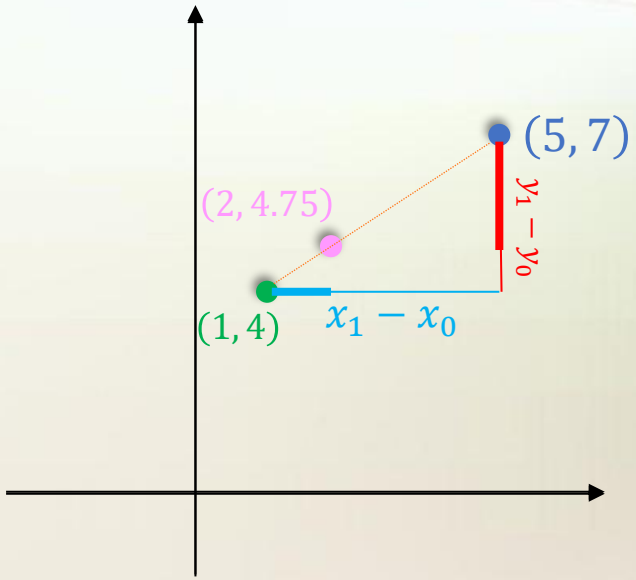
$$(x_M, y_M) = \left(1 + 0.5 * (5 - 1), 4 + 0.5 * (7 - 4)\right) = (3, 5.5)$$

Interpolation: Motivation III

Given two points in the plane*, we can compute any point on the line defined by two points as:

$$(x_u, y_u) = (x_0, y_0) + u \cdot \big((x_1 - x_0), (y_1 - y_0)\big)$$

where u is a constant and (x_0, y_0) is the origination point.



Example. Compute the point that is one-fourth distance away from $(1, 4)$ toward $(5, 7)$.

Solution.

Here, $u = 0.25$.

$$(x_{.25}, y_{.25}) = (1, 4) + \big(0.25 * (5 - 1), 0.25 * (7 - 4)\big) = (2, 4.75)$$

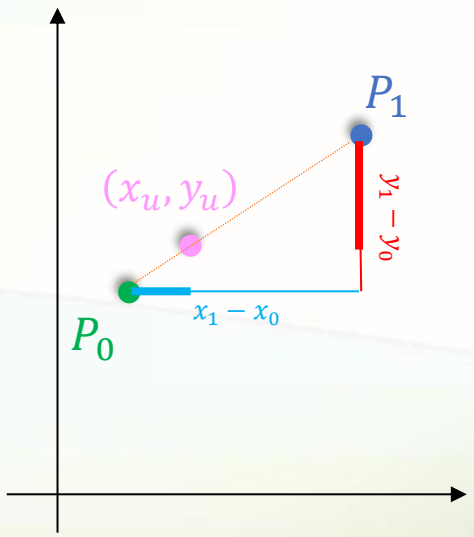
*This can easily be extended to n dimensions.

Linear Interpolation and SMOTE

What may look more familiar is an equivalent expression to our formula.

$$
\begin{aligned}
(x_u, y_u) &= (x_0, y_0) + u \cdot \big((x_1 - x_0), (y_1 - y_0)\big) \\
&= (x_0, y_0) + u \cdot (x_1, y_1) - u \cdot (x_0, y_0) \\
&= u \cdot (x_1, y_1) + (1 - u) \cdot (x_0, y_0) \\
&= u \cdot P_1 + (1 - u) \cdot P_0
\end{aligned}
$$

where P_0 and P_1 are points in the plane and u is a constant.



The idea behind SMOTE:

- Take 2 (real) minority class examples: m_1, m_2.
- Draw randomly $\lambda \in [0,1]$.
- Create a synthetic example S between m_1 and m_2, where λ controls its position along the segment.
- Repeat for many examples to grow the minority class to the desired size

SMOTE Algorithm

SMOTE generates synthetic minority examples by interpolating between existing ones.

Input: Minority example x

1. Find the k nearest neighbors of x among other minority examples (typically $k = 5$)
2. Randomly select one neighbor x'
3. Generate a new synthetic example along the line segment between x and x':

$$x_{new} = x + \lambda \cdot (x' - x), \quad \lambda \in [0,1]$$

4. Repeat until the desired class balance is reached

Observe: due to ***inter***polation, synthetic examples lie **between** real minority examples in feature space and not outside them.

Borderline-SMOTE: focuses synthesis near the decision boundary

Questions

In SMOTE, what is the role of λ?

a) It determines how many neighbors to consider
b) It controls where the synthetic example lies between two minority examples
c) It sets the desired class ratio after oversampling
d) It measures the distance between two minority examples

SMOTE should be applied to:

a) The full dataset before any split
b) The test set only
c) Both training and test sets equally
d) The training set only

Which of the following is a known limitation of SMOTE?

a) It always undersamples the majority class
b) It can generate noisy synthetic examples in regions of class overlap
c) It can only be applied when the minority class has at least 100 examples
d) It only works when $\lambda = 0.5$